

CS301

DATA STRUCTURE AND ALGORITHMS

LECTURE 11: BFS AND DFS ALGORITHMS

Pandav Patel
Assistant Professor

Computer Engineering Department
Dharmsinh Desai University
Nadiad, Gujarat, India

OBJECTIVE

- To understand BFS and DFS algorithms
- To understand differences between iterative and recursive DFS algorithm
- To be able to use BFS for single source shortest distance

OVERVIEW

1 OBJECTIVE

2 BFS ALGORITHM

- Vertex and Edge structures
- Algorithm

3 DFS ALGORITHM

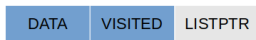
- Iterative
- Recursive

4 BFS FOR SHORTEST DISTANCE

- Vertex and Edge structures
- Algorithm

VERTEX AND EDGE STRUCTURES

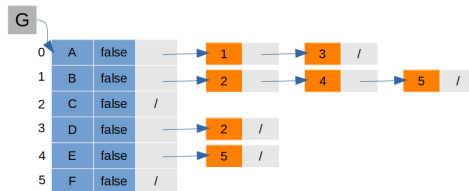
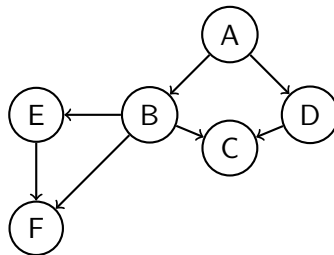
■ Vertex structure



■ Edge structure



- Sample graph and its representation in memory is shown on right
- G is pointer to array representing vertices
- Is vertex structure self referential?



BFS ALGORITHM

CASES

- Single vertex graph
- Null graph with multiple vertices (No edges)
- All vertices are reachable from starting vertex
- All vertices are not reachable from starting vertex

BFS ALGORITHM

STEPS

- 1 Initialize queue with index of starting vertex
 - Mark it visited and print it
- 2 Process all the Vertices in the queue till it is empty
 - 3 Delete a vertex from the queue
 - 4 Process all unvisited neighbours of current vertex
 - Mark visited, print, and insert into queue
- 5 Finish

BFS ALGORITHM

Algorithm: BFS(INDEX)

INDEX indicates starting node for BFS

G is pointer to adjacency list

Structure of nodes is as shown in previous slides

QINSERT and QDELETE is applied on QUEUE

CURR_NODE is temporary integer variable

CURR_EDGE is pointer to EDGE node

EDGE_DEST is pointer to an element of array

1. [Initialize the queue]
 $\text{VISITED}(G + \text{INDEX}) \leftarrow \text{true}$
 $\text{Write}(\text{DATA}(G + \text{INDEX}))$
 $\text{QINSERT}(\text{QUEUE}, \text{INDEX})$
2. [Process nodes from queue till its empty]
 repeat thru step 4 while queue is not empty
3. [Delete a node from queue]
 $\text{QDELETE}(\text{QUEUE}, \text{CURR_NODE})$
 $\text{CURR_EDGE} \leftarrow \text{LISTPTR}(G + \text{CURR_NODE})$

4. [Process all edges outgoing from current node]
 While $\text{CURR_EDGE} \neq \text{NULL}$ do
 $\text{EDGE_DEST} \leftarrow G + \text{DEST}(\text{CURR_EDGE})$
 If $\text{VISITED}(\text{EDGE_DEST}) = \text{false}$ then
 $\text{VISITED}(\text{EDGE_DEST}) \leftarrow \text{true}$
 $\text{Write}(\text{DATA}(\text{EDGE_DEST}))$
 $\text{QINSERT}(\text{QUEUE}, \text{DEST}(\text{CURR_EDGE}))$
 $\text{CURR_EDGE} \leftarrow \text{NEXT}(\text{CURR_EDGE})$
5. [Finished]
 return

BFS ALGORITHM

DISCUSS

- 1 Before processing (printing) a vertex we are checking if it is already visited (if condition in step 4). This will prevent infinite loop
- 2 We are printing (processing) a vertex *before* inserting it to the queue
- 3 What if all vertices are not reachable from starting vertex?
- 4 Will this algorithm work for single vertex graph?

DFS ALGORITHM

CASES

- Single vertex graph
- Null graph with multiple vertices (No edges)
- All vertices are reachable from starting vertex
- All vertices are not reachable from starting vertex

DFS ALGORITHM (ITERATIVE)

STEPS

- 1 Initialize Stack with index of starting vertex
- 2 Process all the Vertices in the stack till its empty
 - 3 Pop a vertex from the stack and process it
 - If already visited, go to step 2
 - Otherwise, mark it visited and print it
 - 4 Push all unvisited neighbours of current vertex to the stack
- 5 Finish

DFS ALGORITHM (ITERATIVE)

Algorithm: DFS(INDEX)

INDEX indicates starting node for DFS

G is pointer to adjacency list

Structure of nodes is as shown in previous slides

PUSH and POP is applied on STACK

CURR_NODE is temporary integer variable

CURR_EDGE is pointer to EDGE node

EDGE_DEST is pointer to an element of array

1. [Initialize the stack]
 PUSH(STACK, INDEX)
2. [Process nodes from stack till its empty]
 repeat thru step 4 while stack is not empty
3. [Pop a node from stack and process it]
 POP(STACK, CURR_NODE)
 If VISITED(G + CURR_NODE) = true then
 go to step 2
 VISITED(G + CURR_NODE) $\leftarrow$ true
 Write(DATA(G + CURR_NODE))
 CURR_EDGE $\leftarrow$ LISTPTR(G + CURR_NODE)

4. [Add all edges outgoing from current node to stack]
 While CURR_EDGE $\neq$ NULL do
 EDGE_DEST $\leftarrow$ G + DEST(CURR_EDGE)
 If VISITED(EDGE_DEST) = false then
 PUSH(STACK, DEST(CURR_EDGE))
 CURR_EDGE $\leftarrow$ NEXT(CURR_EDGE)
5. [Finished]
 return

DFS ALGORITHM (ITERATIVE)

DISCUSS

- 1 Before pushing a vertex we are checking if it is already visited (*if* condition in step 4). What if we remove *if* condition and push vertex without checking if it is already visited? It works fine, as *if* condition in step 3 checks if it has already been visited before processing (printing) it
- 2 We are printing (processing) a vertex *after* popping it from the stack.
- 3 What if all vertices are not reachable from starting vertex?
- 4 Will this algorithm work for single vertex graph?

DFS ALGORITHM (RECURSIVE)

STEPS

- 1 If vertex received is already visited then return. Otherwise process it (mark it visited and print it)
- 2 Call recursive DFS algorithm for all the adjacent vertices which are not already visited
- 3 Finish

DFS ALGORITHM (RECURSIVE)

Algorithm: DFS_RECURSIVE(INDEX)

INDEX indicates starting node for DFS

G is pointer to adjacency list

Structure of nodes is as shown in previous slides

CURR_EDGE is pointer to EDGE node

EDGE_DEST is pointer to an element of array

1. [Process vertex represented by INDEX]

 If VISITED(G + INDEX) = true then

 return

 VISITED(G + INDEX) $\leftarrow$ true

 Write(DATA(G + INDEX))

2. [Recursively call for all adjacent nodes]

 CURR_EDGE $\leftarrow$ LISTPTR(G + INDEX)

 While CURR_EDGE $\neq$ NULL do

 EDGE_DEST $\leftarrow$ G + DEST(CURR_EDGE)

 If VISITED(EDGE_DEST) = false then

 DFS_RECURSIVE(DEST(CURR_EDGE))

 CURR_EDGE $\leftarrow$ NEXT(CURR_EDGE)

3. [Finished]

 return

- Can we remove *if* condition and *return* statement from step 1?
- Can we remove *if* condition from step 2 and invoke recursive call without condition?
- Can we remove both?

DFS ALGORITHM (RECURSIVE)

DISCUSS

- 1 We are printing (processing) a vertex *before* recursive calls for the adjacent (unvisited) vertices.
 - In iterative call we were processing a vertex after popping it. Then in recursive algo, why are we processing it before recursive call for the adjacent vertices?
- 2 Please note that INDEX, CURR_EDGE and EDGE_DEST are local variables, and will have a separate copy for each entry on call stack.
 - Edges before CURR_EDGE have been processed and edges after it are remaining
 - In iterative DFS, this information was not retained. So if we do not push all adjacent vertices on the stack then each time backtracking reaches a vertex, we will have to check all its adjacent vertices. Which will increase time complexity.
 - Can we save on stack an edge pointer before which all edges are processed and after which all edges are remaining. If we do so then we do not need to push all adjacent vertices on stack.
- 3 Will this algorithm work for graph having single vertex?

BFS FOR SHORTEST DISTANCE: VERTEX AND EDGE STRUCTURES

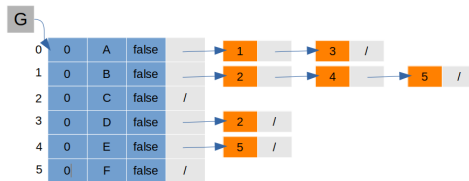
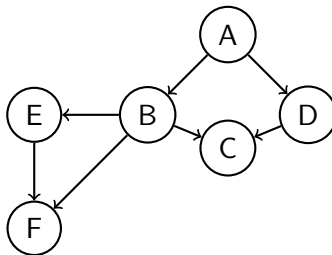
■ Vertex structure

HOPS	DATA	VISITED	LISTPTR
------	------	---------	---------

■ Edge structure

DEST	NEXT
------	------

- Sample graph and its representation in memory is shown on right
- G is pointer to array representing vertices



BFS ALGORITHM FOR SINGLE SOURCE SHORTEST DISTANCE

Algorithm: BFS_HOPS(INDEX)

INDEX indicates starting node for BFS

G is pointer to adjacency list

Structure of nodes is as shown in previous slides

QINSERT and QDELETE is applied on QUEUE

CURR_NODE is temporary integer variable

CURR_EDGE is pointer to EDGE node

EDGE_DEST is pointer to an element of array

DIST is temporary integer to remember hops

1. [Initialize the queue]
 $\text{VISITED}(G + \text{INDEX}) \leftarrow \text{true}$
 $\text{HOPS}(G + \text{INDEX}) \leftarrow 0$
 QINSERT(QUEUE, INDEX)
2. [Process nodes from queue till its empty]
 repeat thru step 4 while queue is not empty
3. [Delete a node from queue]
 QDELETE(QUEUE, CURR_NODE)
 $\text{CURR_EDGE} \leftarrow \text{LISTPTR}(G + \text{CURR_NODE})$
 $\text{DIST} \leftarrow \text{HOPS}(G + \text{CURR_NODE}) + 1$

4. [Process all edges outgoing from current node]
 While $\text{CURR_EDGE} \neq \text{NULL}$ do
 $\text{EDGE_DEST} \leftarrow G + \text{DEST}(\text{CURR_EDGE})$
 If $\text{VISITED}(\text{EDGE_DEST}) = \text{false}$ then
 $\text{VISITED}(\text{EDGE_DEST}) \leftarrow \text{true}$
 $\text{HOPS}(\text{EDGE_DEST}) \leftarrow \text{DIST}$
 QINSERT(QUEUE, $\text{DEST}(\text{CURR_EDGE})$)
 $\text{CURR_EDGE} \leftarrow \text{NEXT}(\text{CURR_EDGE})$
5. [Finished]
 return

- This algorithm is similar to BFS, changes are marked in red

